



College of Engineering

Department of Software Engineering

Selected Topics in Software Engineering (STSE – SWEG5110)

API Testing & Monitoring System

- 1) Abduselam Sultan ----- ETS0024/14
- 2) Abel Tessema ----- ETS0027/14
- 3) Abel Maireg ----- ETS0028/14
- 4) Abel Mulat ----- ETS0029/14
- 5) Abel Seyoum ----- ETS0032/14
- 6) Abel Bogale ----- ETS0035/14

Submitted to – Instructor Kibrom G.

Submitted on – Tuesday May 5, 2026

API Observability Platform

A full-stack API testing and monitoring system for synthetic checks, SLA reporting, Prometheus metrics, and Grafana dashboards.

Runtime	Testing	Observability	Delivery
Node.js + Express	Postman, Newman, Jest, k6	Prometheus + Grafana	Docker Compose + CI

Executive Summary

The API Observability Platform is a miniature production-style monitoring environment for API reliability work. It combines a realistic Express service, synthetic API tests, Prometheus metrics, Grafana visualization, load testing scripts, and Docker-based local orchestration.

Project outcome

A single-command local stack that demonstrates API contract validation, SLA measurement, observability instrumentation, dashboard provisioning, and alert simulation.

System Architecture

The system follows a DevOps-friendly topology: clients and synthetic tests exercise the API, Express emits runtime metrics, Prometheus scrapes the metrics endpoint, and Grafana presents operational health, performance, and reliability signals.

Layer	Component	Responsibility
API	Node.js + Express	Serves /health, /users, /posts, and /metrics with logging, timing, and error handling middleware.
Testing	Postman, Newman, Jest	Validates status codes, schemas, content, response-time SLA, and saves JSON run metrics.
Metrics	prom-client + Prometheus	Exposes counters, histograms, gauges, uptime, request volume, latency, and error counts.
Visualization	Grafana	Provisioned dashboard with health, performance, and reliability rows.
Delivery	Docker Compose + GitHub Actions	Runs the local stack and provides CI smoke checks for API and metrics readiness.

Implementation Scope

Area	Implemented assets	Portfolio signal
Express API	Realistic user and post endpoints, health check, structured errors, request IDs, response timing	Shows backend API design and operational middleware.
Synthetic testing	Postman collection, Newman runner, Jest contract tests, generated SLA JSON	Shows automated validation and test result normalization.
Observability	Prometheus /metrics endpoint, scrape config, alert rules, Grafana dashboard JSON	Shows SRE-style metrics thinking and dashboard design.
Operations	Docker Compose, Dockerfile, k6 stress and spike scripts, CI workflow	Shows local production simulation and automation discipline.

Testing and SLA Strategy

The testing layer blends API contract checks with synthetic monitoring semantics. Postman tests use explicit SLA language, while Newman converts each run into normalized metrics that can be reviewed independently of the CLI output.

- SLA target: Response time under 300ms for core synthetic requests.
- Coverage: GET /health, GET /users, GET /posts, and POST /posts.
- Validation: status codes, response schema, response content, response time, and generated metrics artifact shape.
- Artifacts: metrics/newman/latest-run.json and metrics/newman/latest-summary.json.

Dashboard Design

Dashboard row	Panels	Operational question answered
System Health	Uptime, average latency, total requests, SLA compliance	Is the service currently healthy and meeting its stated SLA?
Performance	Response time trend, endpoint latency comparison, requests per second	Where is traffic or latency shifting over time?
Reliability	Error rate, failed requests, recent failures table	What is failing, how often, and on which route/status combination?

Local Runbook

The project is designed for minimal local setup.

Command	Purpose
npm install	Install Node dependencies.
docker compose up --build	Start API, Prometheus, and Grafana.
npm test	Run Jest API and metrics artifact tests.
npm run test:postman:metrics	Run Newman synthetic SLA suite and save JSON metrics.

Validation Summary

- Jest suite passed with 7 tests covering API contracts, error handling, Prometheus output, and example metrics format.
- Newman synthetic suite passed with 4 requests, 16 assertions, and 0 failures.
- Generated Newman summary reported 100% SLA compliance in the latest local run.
- Docker Compose configuration was parsed successfully with API, Prometheus, Grafana, networks, and persistent volumes.